Deterministic approach for Bridging fault detection in Peres-Fredkin and Toffoli based Reversible circuits

Nagamani A N[#1], Abhishek B[#2], Vinod Kumar Agrawal[*3]
[#]Department of Electronics and Communication Engineering, PES Institute of Technology
Bangalore, India
[*]Director CORI, Professor, Information Science and Engineering, PES Institute of Technology
Bangalore, India
[1]nagamani@pes.edu [2]abhishekbudavarthy@gmail.com [3]vk.agarwal@pes.edu

***Abstract*— Reversible logic has wide applications in the field of quantum computing, low-power design, nanotechnology, optical information processing, bioinformatics etc. Due to the reversible nature of the circuit, it is conventionally easy to test the reversible circuit compared to irreversible circuit but with its own complexity. Various fault models are defined in the literature for reversible circuit. In this paper, we propose a deterministic ATPG algorithm to generate complete test-set for intra-level single and multiple bridging faults in a reversible circuit designed with family of Toffoli, Peres and Fredkin gates. The analysis of results shows that the proposed algorithm has 100% fault coverage and reduction of test set of average of 40% compared to existing approaches in literature.**

Keywords*— *Bridging Fault; Reversible circuits; Deterministic ATPG; Complete test set;

I. INTRODUCTION

Demands for portable and ultra-low power gadgets have dropped the size of transistor to nano-meter scale. Since there is a constraint on the minimum size of a transistor, there is a need for an alternative for low power design. Reversible logic fulfills the above requirement and also finds its applications in the fields of quantum computing, bioinformatics, cryptography, optical computing and nanotechnology etc. authors in [1] proved that, if the computation is irreversible, there is a loss of kTln2 joules of energy in terms of heat for loss of 1-bit of information, where k is a Boltzmann constant and T is the absolute operating temperature. Authors in [2] proved that if the computation is reversible, the dissipation of heat in order of kTln2 joules can be eliminated. A digital circuit is said to be reversible, if the number of input lines is equal to the number of output lines with unique mapping between inputs and outputs [3]. Digital circuits designed using reversible gates should be fan-out free [4].

Fault is a physical disturbance which alters the functionality of the circuit. Since there is huge number of fault space in a given circuit, it is tedious to detect all the faults using a single method. To detect these faults there is need of logical fault model hence various fault models are defined in literature. Some of the popular fault models proposed in the reversible literature are Stuck-at-Fault, Missing Gate Fault, Bridging Fault and Cross point Faults. A fault is said to be detectable, if for any input test vector, the output of a faulty circuit is different from the golden (fault free) output. Complete Test Set is a set of minimal test vectors that detect all possible faults in a given circuit. It is tedious to manually generate all the test vectors if the circuit complexity is high. Hence there is a need of efficient algorithms that automatically generate complete test set for a given fault model. In this paper we propose a deterministic ATPG algorithm which is developed on the basis of set cover approach where the minimal test set is chosen from superset of test vectors. This approach is validated for various benchmark circuits and also few circuits designed with family of Toffoli, Peres, and Fredkin gates to generate complete test set which can detect all the intra level Bridging Faults in a given reversible circuit.

This paper is structured as follows. Section II deals with the detailed discussion on importance, characteristics and basics of Reversible logic, motivation behind the proposed work, basics of Bridging Fault model and investigates the previous researches on existing test generation algorithms to detect Bridging Faults. The proposed ATPG algorithm to detect all the intra level Bridging Faults is explained in Section III in detail with the help of pseudo code and applying it to a benchmark circuit. The algorithm discussed in section III is compared with the existing test pattern generation algorithms and results are tabulated for various benchmark circuits in Section IV. Conclusion of the paper and future work is discussed in Section V and reference follows.

II. PRELIMINARIES

A. Reversible Logic

Reversible circuits compute bijective functions so, the input of the circuit can be restored from its output. Hence there is no loss in the information. As the numbers of input and output lines are same, the output of a circuit will be a permutation of input. A reversible circuit is designed with reversible gates.

There are various standard reversible gates such as Toffoli (k-CNOT), Fredkin and Peres gates. Reversible gates are represented as an n x n gate where n is the number of input and output lines. A k-CNOT gate is (k+1) x (k+1) gate where k indicates the number of control lines, Fredkin and Peres gates are 3 x 3 gates. Reversible circuits are characterized based on parameters like: Quantum cost (QC), total cost in terms of primitive gates, Delay (Δ), number of primitive gates

978-1-4799-7849-6/15/$31.00 ©2015 IEEE

in the critical path, Garbage outputs (GO), number of unused output of the circuit, Ancilla Inputs (AI), total number of constant inputs used and Gate count (GC), number of reversible gates used to realize a circuit. Fig.1 shows the symbols of standard reversible gates. Toffoli (2-CNOT) and Fredkin gates have quantum cost of 5 and delay of 5Δ [5]. Peres gate have quantum cost of 4 and a delay of 4Δ [6].

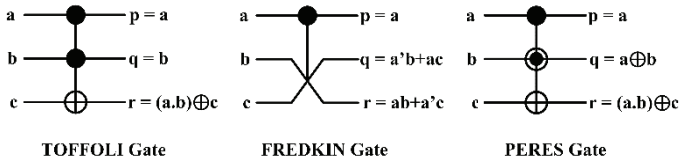


Fig. 1. Symbols of standard Reversible gates

Design of reversible circuits deals with the effective realization of a concept using standard reversible gates. Analysis deals with validation of the circuit designed using reversible gates. Research on synthesis of reversible circuits using k-CNOT gates has been reported in literature BDD [15], ESOP [16] and Transformation [9] based synthesis. Paper [8] introduces a tool to synthesize the reversible circuits using n-Fredkin and n-Peres gates. Authors in [9] propose a transformation based synthesis algorithm to construct the reversible circuits using Fredkin gate. In literature the test generations for circuits designed with Toffoli gates are discussed but circuits designed with standard reversible gates are not discussed. With this motivation, this is a first attempt in generation of Complete test set to detect Bridging Faults in k-CNOT, Fredkin and Peres based reversible circuits.

B. Bridging Fault Model

An unintentional short or coupling between two or more lines of a circuit results in a Bridging Fault. The shorted lines will have the same value based on wired AND or wired OR logic at the connection. Most of the Bridging faults are evident at the input and output lines and at each level of the circuit [10]. There are different types of Bridging Fault based on type of the wired logic subsumed, based on location of the fault and based on the number of lines involved in a fault.

Based on wired logic property, faults can be AND and OR Bridging Fault. If the value of two shorted lines is decided by using the wired-AND logic then the fault is categorized as AND Bridging Fault. If the value of two shorted lines is decided by using the wired-OR logic then the fault is categorized as OR Bridging Fault. The effect of AND and OR Bridging Fault on a circuit is illustrated in Fig. 2.

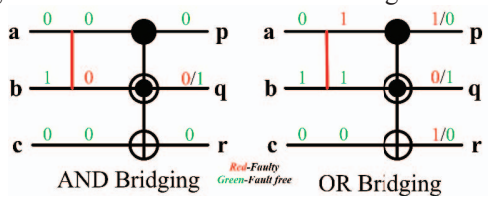


Fig. 2. Illustration of AND Bridging and OR Bridging Fault

Based on the location of the fault, the faults could be at same level or at a different level. Intra level Bridging Fault occurs if two shorted lines are at the same level of the circuit. And inter level Bridging Fault occurs if two shorted lines are at the different level of the circuit. Based on number of lines involved in a fault, faults can be categorized as single and multiple Bridging Fault. If only two lines are involved in a fault then, the fault is categorized as a single Bridging Fault, else the fault is categorized as a multiple Bridging Fault. In order to detect any fault in the reversible circuits, both fault activation and fault propagation is to be performed. To activate a Bridging Fault, both the lines should have an opposite values on their input signals [10] and we need to select a test vector such that the corresponding fault is activated and the fault is propagated to the primary outputs. Since the implementations of reversible gates are still at the research stage, we are analyzing with the post implementation scenario of reversible gates. For an N line and L level circuit, there are (L+1) levels at which Bridging Faults could occur. At each level, NC_2 Bridging Faults are possible. Totally there are NC_2*(L+1) single intra level Bridging faults in a given circuit [10].

C. Literature Survey

Using only the Stuck-at-Fault model, all faults in a reversible circuit cannot be captured. Single and Multiple Missing gate, Partial Missing gate and Repeated Missing gate fault models [11], Bridging Fault model [10], Cross-point fault model [12] has been proposed which can capture most of the physical faults that might occur during the implementation of these reversible circuits in various implementation techniques.

Authors in [4] propose a simple method to generate minimum test vectors to detect all possible intra level Bridging Faults in the reversible circuits designed using only Toffoli gates. A DFT (Design-For-Test) method based test vector generation algorithm is proposed in [10]. This algorithm finds the test vectors to detect all the intra level Bridging Faults and Stuck-at-Faults for Toffoli based reversible circuits at the cost of an extra circuitry to the original circuit. Paper [7] describes a method for testing Bridging Faults in AND-EXOR based k-CNOT gates. k-CNOT gates are represented using irreversible AND and EXOR gates to detect all intra level Bridging Faults. This method [7] requires '3n + log2p +2' (n is the number of gates and p is the number of input lines of each 'p' gate) test vectors to detect the faults. In [13], a polynomial time algorithm is described to generate universal test vectors to detect all intra level single and multiple input Bridging Faults and input Stuck-at-Faults for circuits containing only Toffoli gates. This algorithm detects the faults using 'n' test vectors. The same author proposes a novel method to detect Bridging Faults in Reversible circuits using the properties of unitary matrix. All the Reversible gates are represented as a unitary matrix and test vectors are generated by applying shifting operation on the matrix. Author proves that 'n/2' (n is the number of input lines) number of test vectors is required to detect all intra level single and multiple Bridging Faults [3].

The algorithm proposed in this work is validated for various reversible benchmark circuits and parameters like number of lines, number of gates, total number of detected faults, number of test vectors and time required to generate the complete test set are tabulated for a given reversible circuit and compared with the existing results in literature.

III. Proposed ATPG Algorithm

A. Algorithm 1: Proposed algorithm for Bridging Fault detection

Inputs: Evaluate the number of lines (*N*) and number of levels (*L*)
Output: Complete test set for all single and multiple bridging faults.

```
Step 1: Initialize 'fault_table' to empty array
    for i = 0 to 2^N-1
        Initialize 'fault_array' to empty array
        input = N bit binary equivalent of 'i'
        for every single pair of 'input' lines
            if both lines have same binary value
                fault = 0
            else
                fault = 1
            endif
            update 'fault' to 'fault_array'
        endfor
Step 2: update output of the circuit at each level for the given 'input'.
    for j = 1 to L
        output =j^th level output of the circuit for a given 'input'
            for every single pair of 'output' lines
             check for redundant faults if any
                if both lines have same binary value
                fault = 0
            else
                fault = 1
            endif
                update 'fault' to 'fault_array'
            endfor
        endfor
        update 'fault_array' to i^th row of 'fault_table'
    endfor
Step 3: vectorspace = set of all the elements from 0 to 2^N-1
    for r = 1 to 2^N
        Find all s = 2^N C_2 combinations in 'vectorspace'
        Initialize testvector to empty array
        for k = 1 to s
    Evaluate whether the k^th set covers all the faults in 'fault_table'
            if (true)
                k^th set is one of the test set among 's' sets
                update kth set to 'testvector' array
            endif
        endfor
    if 'testvector' is an empty array
            continue
        else
            break
        endif
    endfor
```

Final 'testvector' contains minimum test set which detects all intra level bridging faults in the given circuit.

B. Illustration of proposed algorithm using an example 1

Consider a reversible benchmark circuit 3_17tc as shown in Fig. 3.

Inputs: 3_17tc is provided as an input to the algorithm.
The circuit has N = 3 and L = 6.

Step 1: Variables '*fault_table*' and '*fault_array*' is initialized to an empty array. Now we need to find all detectable intra level Bridging Faults at every level of the circuit. Fig. 4 represents different levels involved in the circuit.

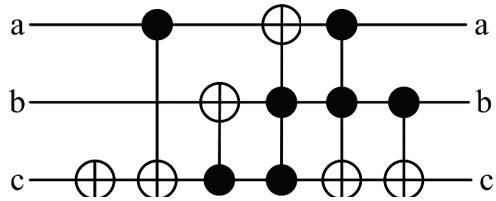


Fig. 3. 3_17tc Reversible Benchmark circuit

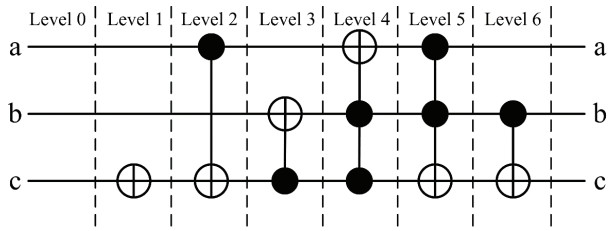


Fig. 4. Level Compaction for 3_17tc Reversible circuit

For a binary input of [0 0 0], algorithm finds all intra level input Bridging Faults at level 0. Totally ${}^{8}C_1$ = 3 single Bridging Faults are possible. For each pair of input lines, find the value of '*fault*'. Since all the inputs are logical 0's, this input cannot detect any bridging fault at input level or level 0. Hence the value of '*fault*' for all 3 faults are 0's. Variable '*fault_array*' will be updated to [0 0 0].

Step 2: Consider [0 0 0] as the input to the circuit and find the output of the circuit at each level. The output of the circuit at each gate is shown in Table I.

TABLE I. Output Of The Circuit For Input Vector [0 0 0]

Each Level output for input vector [0 0 0]						
L0	*L1*	*L2*	*L3*	*L4*	*L5*	*L6*
0	0	0	0	1	1	1
0	0	0	1	1	1	1
0	1	1	1	1	0	1

For each of the output combination, repeat step 3 to determine '*fault*' and update the '*fault_array*'. Finally '*fault_array*' is assigned to first row of the '*fault_table*'. For the input vector [0 0 0], the first row of '*fault_table*' is as shown in Table II. As there are 3 faults at each level of the circuit, the table is filled by considering Bridging Faults in the order of (1,2), (1,3) and (2,3) lines at every level.

TABLE II. Contents Of '*fault_table*' For Input Vector [0 0 0]

B^0			B^1			B^2			B^3			B^4			B^5			B^6		
0	0	0	0	1	1	0	1	1	1	1	0	0	0	0	0	1	1	0	0	0

The above steps are repeated for all combinations of input and '*fault_array'* is updated to corresponding row of '*fault_table'*. The complete '*fault_table'* is shown in Table III.

Step 3: This step finds all possible minimum test set to detect intra level Bridging Fault. Variables '*vectorspace*' is initialized an array containing the elements from 0 to (2^N-1) and '*testvector*' is initialized to an empty array. Find all ${}^{2^N}C_r$ combinations in '*vectorspace*' for 'r' varying from 1 to 2^N. In this example, N=3 and '*vectorspace*' is [0 1 2 3 4 5 6 7]. Each element in '*vectorspace*' equates to an entire row of '*fault_table'*. For r = 1, there are ${}^8C_1$ = 8 combinations. Each entry in '*fault_table'* indicates whether the fault is detectable or not. If the entry is a 1 then, the corresponding bridging fault is detectable; else the fault is undetectable. For each combination, find whether the corresponding set detect all the faults in '*fault_table'*. It is observed that none of the single input vector detects all the Bridging Fault so, '*testvector*' is still an empty array. We increment r and repeat the process again. For r = 2, we get ${}^8C_2$ = 28 combinations. It is found that out of 28 combinations only one combination i.e. [2, 3] detect all Bridging Faults. Hence this combination is treated as a test vector and is updated to the variable '*testvector*'. Since variable '*testvector*' is not an empty array, algorithm gets terminated. The test set for the circuit is 2, [0 1 0] and 3, [0 1 1] . The detailed fault table for the example is shown in Table IV.

TABLE III. COMPLETE FAULT TABLE FOR 3_17TC BENCHMARK CIRCUIT

Input			Bridging Faults at each level																				
a	*b*	*c*	B^0			B^1			B^2			B^3			B^4			B^5			B^6		
0	0	0	0	0	0	0	1	1	0	1	1	1	1	0	0	0	0	0	1	1	0	0	0
0	0	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	1	0	1	0	1	1	1	0	1	1	0	0	1	1	0	1	1	0	1	1	0	1	1
0	1	1	1	1	0	1	0	1	1	0	1	1	0	1	1	0	1	1	0	1	1	1	0
1	0	0	1	1	0	1	0	1	1	1	0	1	1	0	1	1	0	1	1	0	1	1	0
1	0	1	1	0	1	1	1	0	1	0	1	0	0	0	1	1	0	1	1	0	1	0	1
1	1	0	0	1	1	0	0	0	0	1	1	0	1	1	0	1	1	0	0	0	0	1	1
1	1	1	0	0	0	0	1	1	0	0	0	1	0	1	1	0	1	1	0	1	1	0	1

TABLE IV. OUTPUT OF THE CIRCUIT FOR INPUT VECTOR [0 1 0], [0 1 1]

Each Level output for input vector [0 1 0]							Each Level output for input vector [0 1 1]						
L0	*L1*	*L2*	*L3*	*L4*	*L5*	*L6*	*L0*	*L1*	*L2*	*L3*	*L4*	*L5*	*L6*
0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	0	0	0	0	1	1	1	1	1	1	1
0	1	1	1	1	1	1	1	0	0	0	0	0	1

By analyzing the output at each level, it is possible to determine which of the Bridging Faults are detectable. Fig 5 and 6 shows pictorial representation of fault coverage by the inputs 2 [0 1 0] and 3 [0 1 1]. The proposed algorithm also evaluates the circuit for undetectable or redundant faults with 100% fault coverage.

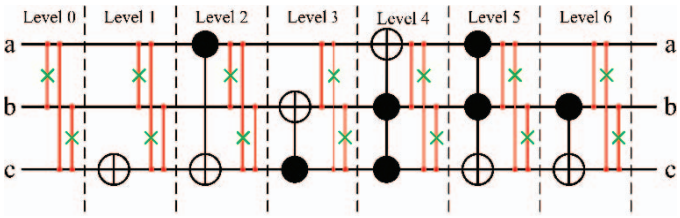


Fig. 5. Fault Coverage by Input Vector 2 [0 1 0] for 3_17tc

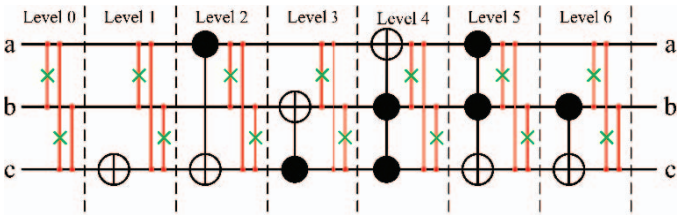


Fig. 6. Fault Coverage by Input Vector 3 [0 1 1] for 3_17tc

C. Illustration of proposed algorithm for example 2

Example 1 illustrates the proposed algorithm for a Toffoli based reversible circuit. Example 2 considers an arbitrary reversible circuit designed for illustration purpose using k-CNOT, Fredkin and Peres gates as shown in Fig. 7. All the intra level Bridging faults involved in the circuit is shown in Fig. 8.

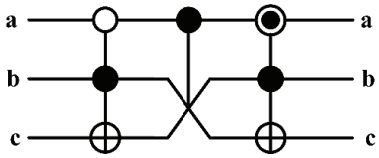


Fig. 7. An arbitrary reversible circuit using standard Reversible gate

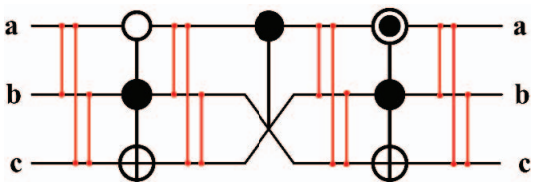


Fig. 8. An arbitrary reversible circuit with Intra level Bridging Faults

For the circuit, inputs are N = 3 and L = 3. For a binary input [0 0 0], update the 'fault_array' by repeating the procedure discussed in example 1. Step 2 evaluates output of the circuit at each level. Similarly all possible bridging faults at each level is evaluated to compute 'fault_array' and update 'fault_array' to the first row of 'fault_table'. It is found that the input vector [0 0 0] cannot detect any fault. Repeat step 2 for all the combinations of input and update the 'fault_array' to the corresponding row of 'fault_table'. The complete 'fault_table' for the circuit is shown in Table V. Perform step 3 to determine minimal test set to detect all intra level single and multiple Bridging Faults. Again it is found that none of single test vector detect all the faults and out of ${}^8C_2$ = 28 combinations, 3 test sets detects all the faults. We get {(1,4), (4,6) and (5,6)} as the test vectors. Let us manually verify whether the test set {4,6} covers all the faults. The output of the circuit at each level for inputs 4, [1 0 0] and 6, [1 1 0] is

shown in Table VI. Table VII tabulates which of the Bridging Faults are covered by the input vectors 4, [1 0 0] and 6, [1 1 0].

Authors in [10] proved that multiple input bridging faults are detectable if and only if single input bridging faults for any subset of input lines are detected and are undetectable if none of the subset single input faults are undetectable. This concept could be extended to all the levels of a circuit. Hence in this algorithm we limit our test set generation to only intra level single Bridging Faults and the obtained test set could also detect all intra level multiple Bridging Faults and hence enhancing the fault coverage. The multiple Bridging Faults/ lines could be split into individual single Bridging Faults/ lines. As the generated test set covers all single Bridging Faults, multiple Bridging Faults can inherently be detected using the same test set. This algorithm considers NC_r*(L+1) single intra level Bridging faults with the coverage of (L+1)*$\sum_{r=2}^{N} NC_r$ number of single and multiple intra level Bridging Faults. In the entire analysis, it is assumed that the circuit has utmost one fault instance at any given time.

TABLE V. COMPLETE FAULT TABLE FOR EXAMPLE 2 CIRCUIT

Input			Bridging Faults at each level											
a	*b*	*c*	B^0			B^1			B^2			B^3		
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1	1	0	1	1
0	1	0	1	0	1	1	1	0	1	1	0	0	0	0
0	1	1	1	1	0	1	1	0	1	1	0	1	1	0
1	0	0	1	1	0	1	1	0	1	1	0	1	1	0
1	0	1	1	0	1	1	0	1	0	1	1	1	1	0
1	1	0	0	1	1	0	1	1	1	0	1	1	0	1
1	1	1	0	0	0	0	0	0	0	0	0	1	0	1

TABLE VI. OUTPUT OF EXAMPLE-2 CIRCUIT FOR INPUT [1 0 0], [1 1 0]

Each Level output for input vector [1 0 0]				Each Level output for input vector [1 1 0]			
L0	*L1*	*L2*	*L3*	*L0*	*L1*	*L2*	*L3*
1	1	1	1	1	1	1	1
0	0	0	0	1	1	0	0
0	0	0	0	0	0	1	1

TABLE VII. FAULT COVERAGE FOR INPUT [1 0 0] AND [1 1 0] TO EXAMPLE-2 CIRCUIT

Inputs	Bridging Fault covered at each level			
	L0	*L1*	*L2*	*L3*
[1 0 0]	(1,2), (1,3)	(1,2), (1,3)	(1,2), (1,3)	(1,2), (1,3)
[1 1 0]	(1,3), (2,3)	(1,3), (2,3)	(1,2), (2,3)	(1,2), (2,3)

IV. RESULTS AND DISCUSSIONS

The proposed algorithm is applied to various Benchmark circuits and the results are compared with [4], [10] and [3]. The test pattern generation algorithms to detect intra level single Bridging Faults proposed in [4] and [3] are compared and the results are tabulated in Table VIII and IX respectively. Algorithm to detect intra level single and multiple input Bridging Faults discussed in [3] is compared and results are tabulated in Table X. Table XI shows the results for other Benchmark circuits and few new circuits designed with combination of k-CNOT, Peres and Fredkin gates. The new circuits are shown in fig.9. It is observed that the proposed algorithm covers more faults when compared with the existing methods and on an average there is a 40% reduction in the number of test vectors required to detect all the faults.

Analysis: For a given circuit, the algorithm computes N, number of lines and L, number of gate levels. The methods proposed in [4] has the complexity of $O(NL^2\log_2(N))$, [10] has the complexity of $O(N\log_2 N)$ with the cost of extra circuitry to actual circuit and [3] has the complexity of $O([2^{2N}(1+L)+\sum_{r=1}^{N} {}^{2^N}C_2])$. The proposed ATPG algorithm has the complexity of $O(2^N)$.

TABLE VIII. COMPARISON OF RESULTS WITH [4]

Name of Benchmark circuit	# I/ O	# Faults in [4]	# TV in [4]	# Single Intra level Bridging Faults	Total #detectable Faults	# TV	Time in (s)
ham3tc	3/3	18	3	18	24	2	0.1372
graycode6	6/6	90	9	90	342	3	1.2674
4_49	4/4	78	5	78	143	3	0.2230
ham7tc	7/7	504	22	504	2880	4	14.0229
rd32	4/4	30	6	30	55	3	0.2098
xor5d1	5/5	50	8	50	130	3	0.2944

Note: TV-Test Vectors, N-Number of lines, L-Number of levels in a circuit. Time (s) indicates the time required to generate complete test set.

TABLE IX. COMPARISON OF RESULTS WITH [10]

Name of Benchmark circuit	N-L	# Faults in [10]	# TV in [10]	# Single Intra level Bridging Faults	Total #detectable Faults	# TV	Time in (s)
ham3tc	3-5	24	4	18	24	2	0.13
graycode 6	6-5	84	5	90	342	3	1.26
hwb4-11-23	4-11	120	4	72	132	4	0.27
4_49-12-32	4-12	130	4	78	143	3	0.22
mod5adder-15	6-15	336	5	240	912	4	82.42
ham7tc	7-23	672	5	504	2880	4	14.02
hwb9-1544	1-7	69525	6	55620	775590	7	914.56
hwb10-3631	10-3631	199760	6	163440	3679216	8	1091.23

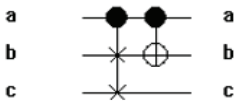
V. CONCLUSIONS

In this paper, we propose an exact ATPG algorithm to generate minimal test vectors to detect all single and multiple

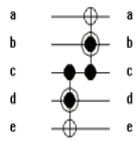
Bridging Faults. Test sets are generated to test the reversible circuits designed using the combination of k-CNOT, Fredkin and Peres gates. The proposed algorithm is coded in MATLAB 2011a on an Intel Celeron processor with 2 GHz processing speed. The generated test vectors are reduced by an average of 40% compared to existing state-of-the-art test generation algorithms for intra level bridging faults. The future work would be to reduce the time complexity of the test generation and extend the algorithm to detect inter level Bridging Faults.

TABLE X. COMPARISON OF RESULTS WITH [3]

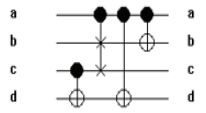
Name of Benchmark circuit	#I/ O	# Faults in [3]	# TV in [3]	# Single Intra level Bridging Faults	Total #detectable Faults	#TV	Time in (s)
6symd2	6/1	90	3	945	21273	5	26.40
9symd2	9/1	352	5	1914	118407	5	255.22
ham7tc	7/7	152	4	504	2880	4	14.02
hwb8-614	8/8	238	4	17220	151905	6	170.33
hwb6-42	6/6	90	3	645	2451	4	110.42
rd73d2	7/3	152	4	945	21273	2	28.30
rd84d1	8/4	238	4	3045	949808	5	1285.2
ham7tc	7/7	152	4	504	2880	4	69.59



(a) pbc_1



(b) pbc_2



(c) rbc

Fig. 9. Some circuits designed with combination of k-CNOT, Peres and Fredkin gates.

TABLE XI. TABULATION OF RESULTS FOR OTHER BENCHMARK CIRCUITS

Name of Benchmark circuit	N-L	# Single Intra level Bridging Faults	Total #detectable Faults	# TV	Time in (s)
2-4 dec	6-3	60	228	3	3.72
2of5d1	6-18	285	1083	4	25.40
3_17tc	3-6	21	28	2	0.17
4b15_1	4-15	96	176	3	0.20
gf2^4mult19-83	12-19	1320	81660	4	76.37
ham3tc	3-5	18	24	2	1.22
hwb4tc	4-17	108	198	3	0.55
mod5d1	5-8	90	234	3	0.97
mspk_4_49_12	4-12	78	143	3	0.53
mspk_4b15g_1	4-15	96	176	4	0.86
mspk_hwb4_12	4-12	78	143	3	0.57
mspk_nth_primes4_14	4-14	90	165	4	0.75
nth_prime3_inc	3-4	15	20	3	0.29
nth_prime4_inc_d1	4-12	78	143	3	0.49
nth_prime5_inc_25_103	5-25	260	676	4	6.08
nth_prime6_inc_55_667	6-55	840	3192	4	116.36
*5mod5_fc	6-10	165	627	3	4.768
*hwb6_fc	6-25	390	1482	5	93.73
*hwb6ps	6-6	1008	14056	4	24.69
*hwb8ps	8-8	2574	159237	5	318.81
*4_49_fc #	4-12	78	143	3	16.48
*ham3_fc #	3-4	15	20	2	0.81
*hwb4_fc #	4-9	60	110	3	16.02
pbc_1$^+$	3-2	9	12	3	0.1985
rbc$^+$	4-4	30	55	3	0.2887
pbc_2$^+$	5-2	30	78	3	0.6748

Note: # indicates that the circuit has redundant faults.* indicates that circuits are designed using combination of k-CNOT, Peres and Fredkin gates.$^+$ some new circuits designed using combination of k-CNOT, Peres and Fredkin gates.

REFERENCES

[1] R. Landauer, ‘‘Irreversibility and heat generation in the computational process’’, *IBM J. Res. Develop*., vol. 5, pp. 183-191, 1961.

[2] C. H. Bennet, “Logical reversibility of computation”, *IBM J. Res. Develop*., vol. 17, no. 6, pp. 525- 532, 1973.

[3] Pradyut Sarkar, Bikromandittya Mondal, Susanta Chakraborty, “Optimal Universal Test Set for Bridging Faults Detection in Reversible Circuit Using Unitary Matrix”, In *RASDAT*,2011.pp 37- 42, IEEE, 2011

[4] Rahaman, Hafizur, Dipak K. Kole, Debesh K. Das, and Bhargab B. Bhattacharya. "Optimum test set for bridging fault detection in reversible circuits." In *Asian Test Symposium, 2007. ATS'07. 16th*, pp. 125-128. IEEE, 2007.

[5] E. Fredkin and T. Toffoli, ‘‘Conservative logic,’’ *Int. J. of Theoretical Physics*, vol. 21, pp. 219-253, 1982.

[6] Peres, A.: Reversible Logic and Quantum Computers. In: Physical Review, A 32: 3266–3276,1985.

[7] Chakraborty, Avik. "Testing of Bridging Faults in AND-EXOR based Reversible Logic Circuits." *arXiv preprint arXiv:1009.5098,* 2010.

[8] James Donald, Niraj K. Jha, “Reversible Logic Synthesis with Fredkin and Peres Gates”, in *ACM journal on Emerging Technologies in Computing Systems (JETC),* Vol. 4, March 2008.

[9] Mathias Soeken, Anupam Chattopadhyay, “ Fredkin-Enable Transformation-based Reversible Logic Synthesis”, In *45th International Symposium on Multiple-Valued Logic (ISMVL),* 2015.

[10] Bubna, Mayur, Nitin Goyal, and Indranil Sengupta. "A DFT methodology for detecting bridging faults in reversible logic circuits." In *TENCON 2007-2007 IEEE Region 10 Conference*, pp. 1-4. IEEE, 2007

[11] I. Polian, T. Fiehn, B. Becker, J.P. Hayes, “A Family of Logical Fault Models for Reversible Circuits”,*14th Asian Test Symposium*, 2005.

[12] Zhong, J.C. Muzio, “Analyzing fault models for Reversible Logic Circuits”,*IEEE Congress on Evol .Computation*, pp. 2422-2427, 2006.

[13] Sarkar, Pradyut, and Susanta Chakrabarti. "Universal test set for bridging fault detection in reversible circuit." In *Design and Test Workshop, 2008. IDT 2008. 3rd International*, pp. 51-56. IEEE, 2008.

[14] Wille, Robert, and Rolf Drechsler. "BDD-based synthesis of reversible logic for large functions." In *Proceedings of the 46th Annual Design Automation Conference*, pp. 270-275. ACM, 2009.

[15] Drechsler, Rolf, Alexander Finder, and Robert Wille. "Improving ESOP-based synthesis of reversible logic using evolutionary algorithms." In *Applications of Evolutionary Computation*, pp. 151-161. Springer Berlin Heidelberg, 2011.